

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

C02: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Scenario based

Code of Conduct:

I ANGEL DEEPIKA M (192521272) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Angel Deepika M

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.

4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

**(10
arks)**

M

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
 - M2: Install drivers
 - M3: Update registry
 - M4: Validate system integrity
- Resources involved:
- R1 = Disk I/O
 - R2 = Kernel lock
 - R3 = Configuration lock
- Recently logs show:
- M1 holds R1, waiting for R2
 - M2 holds R2, waiting for R3
 - M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.

2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10

M

arks) Question 5 — Cloud Server CPU Utilization Analysis A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs Using the formula:

where p = I/O wait probability

$$U = 1 - p^n$$

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10

M

arks)

Question 6 — Hospital Emergency System Process Scheduling

A hospital information system manages:

- P1 – Emergency patient registration (Highest Priority)
- P2 – Laboratory report generation (Medium Priority)
- P3 – Pharmacy billing (Low Priority)

During peak hours, pharmacy billing is delayed while emergency requests continue to arrive.

Tasks

1. Identify the scheduling issue occurring in the system.
2. Recommend a suitable CPU scheduling algorithm.
3. Explain how starvation can be prevented.

4. Discuss how your solution improves response time and fairness.

(10 Marks)

Question 7 — E-Commerce Warehouse Synchronization

An e-commerce warehouse has multiple worker threads updating a shared inventory database simultaneously.

Developers observe:

- Incorrect stock counts
- Duplicate order confirmations
- Frequent data inconsistencies

Tasks

1. Identify the synchronization problem.
2. Explain why a critical section is required.
3. Recommend either semaphores or monitors to solve the issue.
4. Explain how the proposed solution maintains data consistency.

(10 Marks)

Question 8 — Video Streaming Service Thread Management

A video streaming platform creates one thread for every client connection.

During a live sports event:

- 60,000 users connect simultaneously.
- CPU utilisation reaches 95%.
- Memory consumption increases rapidly.
- Video buffering becomes frequent.

Tasks

1. Identify the thread-management problem.
2. Recommend a suitable multithreading model.
3. Suggest two optimisations to improve scalability.
4. Explain how your recommendations improve throughput.

(10 Marks)

Question 9 — Smart Traffic Control Deadlock

A smart city traffic management system controls four intersections.

Resources:

- R1 – North Junction • R2 – East Junction
- R3 – South Junction

- R4 – West Junction

Current situation:

- T1 holds R1 and requests R2.
- T2 holds R2 and requests R3.
- T3 holds R3 and requests R4.
- T4 holds R4 and requests R1.

Tasks

1. Determine whether a deadlock exists.
2. Explain the four necessary conditions for deadlock in this scenario.
3. Recommend one recovery technique.
4. Suggest a redesign that prevents similar deadlocks.

(10 Marks)

Question 10 — University Examination Server Resource Allocation

An online examination server allocates CPU, memory and database connections to student sessions.

At examination time:

- Thousands of students log in simultaneously.
- New resource requests arrive continuously.
- The administrator wants to avoid deadlocks without reducing resource utilisation.

Tasks

1. Explain how Banker's Algorithm can be applied.
2. Describe the information required before granting a request.
3. Explain how the system determines whether it is in a safe state.
4. Discuss one limitation of using Banker's Algorithm in large-scale cloud systems.

(10 Marks)

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marks Evaluation:

Question No.	Understanding of Scenario (20%)	Identification of Key Issues (20%)	Application of Concepts (25%)	Decision Making (20%)	Clarity of Response (15%)	Total Marks(100)
Q1						
Q2						
Q3						
Q4						
Q5						

Q6						
Q7						
Q8						
Q9						
Q10						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

1. Most Suitable Scheduling Algorithm

The most suitable scheduling algorithm is **Priority Scheduling** because the processes have different priority levels.

- **P1 – Payment Gateway Transactions:** High Priority
- **P2 – Seat Availability Checks:** Medium Priority
- **P3 – Ticket Confirmation & Printing:** Low Priority

Priority Scheduling always executes the highest-priority process first, ensuring that critical payment transactions are completed without delay.

2. Justification

Priority Scheduling improves CPU utilisation by allowing the CPU to execute the most important tasks first. Payment transactions are time-critical and must be completed immediately to avoid transaction failures. Once payments are processed, seat availability checks and ticket confirmation are executed.

Advantages:

- Faster payment processing.
- Reduced customer waiting time.
- Better response during heavy booking periods.
- Improved overall system performance.

3. Sample Execution Order

Assume the following burst times:

Process Burst Time Priority

P1	5 ms	High
----	------	------

Process Burst Time Priority

P2 8 ms Medium

P3 10 ms Low

Execution Order:

P1 → P2 → P3

Gantt Chart

0 5 13 23

| P1 | P2 | P3 |

4. Drawback

The main drawback is **starvation**. If high-priority payment requests continue arriving, low-priority processes like ticket confirmation may wait indefinitely.

Solution: Use the **Aging** technique, which gradually increases the priority of waiting processes until they receive CPU time.

Conclusion

Priority Scheduling is the best choice because it executes critical payment transactions first, reduces waiting time, improves CPU utilisation, and provides faster service during peak booking periods.

Question 2 – Deadlock in a Banking Transaction Server

1. Resource Allocation Graph (Text-Based RAG)

A → T1

T1 → B

$B \rightarrow T2$

$T2 \rightarrow C$

$C \rightarrow T3$

$T3 \rightarrow A$

Where:

- **A** = Account Database
- **B** = Customer Info Database
- **C** = Transaction Logs

2. Has a Deadlock Occurred? Justify

Yes, a deadlock has occurred.

Reason:

- **T1** holds **A** and waits for **B**.
- **T2** holds **B** and waits for **C**.
- **T3** holds **C** and waits for **A**.

This forms a circular waiting chain:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

Since each transaction waits for a resource held by another transaction, none of them can proceed.

3. Recommended Technique

Deadlock Prevention

Use a **fixed resource allocation order**.

Example:

$A \rightarrow B \rightarrow C$

Every transaction must request resources only in this order. This eliminates circular waiting and prevents deadlocks.

4. Locking Order Redesign

The redesigned locking sequence should be:

Step 1 : Lock Account Database (A)

↓

Step 2 : Lock Customer Info Database (B)

↓

Step 3 : Lock Transaction Logs (C)

All transactions must follow the same order before execution.

Conclusion

By following a fixed locking order, circular wait is eliminated, preventing deadlocks and ensuring smooth banking transactions.

Question 3 – Mobile OS App Switching Lag

1. Why Does the CPU Remain Idle?

Although many applications are active, most background apps are waiting for I/O operations such as network access, storage access, or user input. During this waiting period, the CPU has no ready process to execute, resulting in high idle time.

2. Is the 20 ms Time Quantum Helping?

No.

With **20 background applications**, a **20 ms quantum** causes frequent context switching. Excessive context switching increases CPU overhead and reduces the time available for actual process execution, leading to lag during app switching.

3. Recommendation

Use **Multilevel Feedback Queue (MLFQ)** scheduling or increase the **time quantum to 40–50 ms**.

Justification:

- Reduces unnecessary context switches.
- Gives interactive applications higher priority.
- Improves CPU efficiency.
- Allows longer execution for CPU-bound tasks.

4. How Does This Improve Performance?

The proposed change:

- Reduces context-switching overhead.
- Improves CPU utilisation.
- Speeds up app switching.
- Increases system throughput.
- Provides a smoother user experience while multitasking.

Conclusion

Using MLFQ or a larger time quantum improves responsiveness, reduces switching lag, and makes better use of CPU resources in a mobile operating system.

Question 4 – Deadlock Risk in an Operating System Update

1. Identify whether this sequence can lead to a deadlock.

Yes, this sequence can lead to a deadlock.

During the OS update, each module holds one resource while waiting for another. Since none of the modules can proceed without acquiring the requested resource, the system may enter a deadlock state.

2. Explain the Circular Wait Condition

Current resource allocation:

- **M1 holds R1 (Disk I/O) and waits for R2 (Kernel Lock).**
- **M2 holds R2 (Kernel Lock) and waits for R3 (Configuration Lock).**
- **M3 holds R3 (Configuration Lock) and waits for R1 (Disk I/O).**

Circular Wait:

M1 → R2

↑ ↓

R1 M2

↑ ↓

M3 ← R3

Or simply:

M1 → M2 → M3 → M1

Since each module waits for a resource held by another, none can continue, resulting in a deadlock.

3. Propose Banker's Algorithm Style Resource Allocation

Banker's Algorithm grants a resource request only if the system remains in a **safe state** after allocation.

Steps:

- Check Available resources.
- Calculate Need = Maximum – Allocation.
- Grant the request only if all modules can complete successfully in some sequence.
- Otherwise, delay the request until resources become available.

This prevents the system from entering an unsafe state and avoids deadlock.

4. Suggest Module Prioritisation

Priority Order:

1. **M1 – Copy Update Files**
2. **M2 – Install Drivers**
3. **M3 – Update Registry**
4. **M4 – Validate System Integrity**

Executing modules in this order reduces waiting time, avoids unnecessary resource contention, and improves CPU utilisation during the update process.

Conclusion

Deadlock is possible due to circular waiting. Applying Banker's Algorithm and assigning a fixed execution priority ensures safe resource allocation, prevents deadlocks, and improves system performance.

Question 5 – Cloud Server CPU Utilisation Analysis

Given

- I/O wait probability (**p**) = **0.8**
- Number of Virtual Machines (**n**) = **7**

CPU Utilisation Formula:

$$\text{CPU Utilisation} = 1 - p^n$$

1. Calculate CPU Utilisation

$$\begin{aligned} &= 1 - (0.8)^7 \\ &= 1 - 0.2097 \\ &= 0.7903 \end{aligned}$$

CPU Utilisation = 79.03%

2. Interpret the Result

A CPU utilisation of approximately **79%** indicates that the server is **well utilised**. The CPU is busy most of the time while still having some spare capacity to handle additional workloads.

This is generally considered an efficient operating condition because the CPU is neither overloaded nor significantly underutilised.

3. Effect of Changing the Number of VMs

Increasing the number of VMs:

- Improves throughput by allowing more processes to execute.
- Better CPU utilisation because waiting times overlap.
- However, too many VMs may increase context switching and reduce performance.

Decreasing the number of VMs:

- Reduces throughput.
- Leaves the CPU idle more frequently.

- Results in poor utilisation of system resources.

4. Two OS-Level Strategies

Strategy 1: Efficient CPU Scheduling

- Use scheduling algorithms such as **Multilevel Feedback Queue (MLFQ)** or **Completely Fair Scheduler (CFS)**.
- These algorithms distribute CPU time efficiently among VMs.

Strategy 2: Asynchronous I/O

- Allow CPU execution while I/O operations are in progress.
- This overlaps computation and I/O, increasing CPU utilisation.

Conclusion

The server has a CPU utilisation of **79.03%**, indicating efficient performance. Proper scheduling and asynchronous I/O can further improve throughput and resource utilisation.

Question 6 – Hospital Emergency System Process Scheduling (10 Marks)

1. Identify the Scheduling Issue

The scheduling issue is **Starvation**.

Emergency patient registrations (**P1**) continuously receive the highest priority, causing lower-priority processes such as pharmacy billing (**P3**) to wait for a long time.

2. Recommend a Suitable CPU Scheduling Algorithm

The most suitable algorithm is **Priority Scheduling with Aging**.

Priority Scheduling ensures that emergency patients receive immediate attention, while the Aging technique gradually increases the priority of waiting processes.

Priority Levels:

Process	Priority
P1 – Emergency Registration	Highest
P2 – Laboratory Reports	Medium
P3 – Pharmacy Billing	Lowest

3. Explain How Starvation Can Be Prevented

Aging prevents starvation by gradually increasing the priority of processes that have been waiting for a long time.

For example:

- Initially, Pharmacy Billing has low priority.
- If it waits for an extended period, its priority increases.
- Eventually, it becomes high enough to receive CPU time.

Thus, every process gets a fair opportunity to execute.

4. How Does the Solution Improve Response Time and Fairness?

Improved Response Time:

- Emergency patient registration is completed immediately.
- Critical medical services are not delayed.

Improved Fairness:

- Aging ensures that laboratory reports and pharmacy billing are eventually executed.

- No process waits indefinitely.

This provides both quick emergency response and fair CPU allocation.

Conclusion

Priority Scheduling with Aging is the best choice for a hospital information system because it guarantees immediate processing of emergency cases while preventing starvation of lower-priority services. It improves response time, fairness, and overall system efficiency.

Question 7 – E-Commerce Warehouse Synchronization

1. Identify the Synchronization Problem

The synchronization problem is a **Race Condition**.

A race condition occurs when multiple worker threads access and modify the shared inventory database simultaneously. Since the threads execute concurrently, the final stock count may become incorrect, leading to duplicate order confirmations and data inconsistencies.

2. Explain Why a Critical Section is Required

A **critical section** is a part of the program where shared resources (inventory database) are accessed or modified.

In this system:

- Multiple threads update stock quantities.
- If two threads update the same item at the same time, one update may overwrite the other.
- This results in incorrect inventory records.

Therefore, only **one thread should access the shared inventory at a time**, ensuring accurate updates.

3. Recommend Semaphores or Monitors

Recommended Solution: Binary Semaphore (Mutex)

A binary semaphore allows only one thread to enter the critical section at a time.

Working:

1. A thread executes **Wait()** before entering the critical section.
2. If the semaphore is available, the thread enters and updates the inventory.
3. After completing the update, it executes **Signal()**, allowing the next thread to proceed.
- 4.

4. Explain How the Solution Maintains Data Consistency

The binary semaphore ensures **mutual exclusion**, meaning only one thread updates the inventory at a time.

This:

- Prevents race conditions.
- Eliminates duplicate order confirmations.
- Maintains correct stock counts.
- Ensures data consistency and reliability.

Conclusion

Using a binary semaphore protects the critical section, prevents simultaneous updates, and ensures accurate inventory management in the e-commerce warehouse.

Question 8 – Video Streaming Service Thread Management

1. Identify the Thread Management Problem

The platform creates **one thread for every client connection**. During a live sports event, 60,000 users connect simultaneously, resulting in:

- Excessive thread creation.
- High memory consumption.
- Increased CPU overhead due to context switching.
- Frequent video buffering.

This approach is not scalable for a large number of users.

2. Recommend a Suitable Multithreading Model

The most suitable model is a **Thread Pool**.

Instead of creating a new thread for every client, a fixed number of worker threads are created in advance. Incoming client requests are assigned to available threads from the pool.

This reduces thread creation overhead and improves performance.

3. Suggest Two Optimisations

(i) Implement Thread Pooling

- Reuses existing threads.
- Reduces memory usage.
- Minimises thread creation time.

(ii) Use Asynchronous (Non-blocking) I/O

- Threads do not wait for network operations.
- One thread can handle multiple client requests efficiently.

4. Explain How the Recommendations Improve Throughput

Thread pooling and asynchronous I/O:

- Reduce CPU overhead.
- Minimise context switching.
- Lower memory consumption.
- Handle more client requests simultaneously.
- Improve scalability during live streaming events.
- Reduce video buffering and increase overall throughput.

Conclusion

A thread pool combined with asynchronous I/O provides better scalability, efficient resource utilisation, and smoother video streaming during heavy user traffic.

Question 9 – Smart Traffic Control Deadlock

1. Determine Whether a Deadlock Exists

Yes, a deadlock exists.

Current allocation:

- **T1** holds **R1** and requests **R2**.
- **T2** holds **R2** and requests **R3**.
- **T3** holds **R3** and requests **R4**.
- **T4** holds **R4** and requests **R1**.

This forms a circular waiting chain, so no process can continue.

2. Explain the Four Necessary Conditions for Deadlock

1. **Mutual Exclusion:** Each junction can be controlled by only one traffic controller at a time.
2. **Hold and Wait:** Each controller holds one junction while requesting another.
3. **No Preemption:** A junction cannot be forcibly taken away from a controller.
4. **Circular Wait:** The waiting sequence is:

$T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T1$

Since all four conditions are satisfied, a deadlock occurs.

3. Recommend One Recovery Technique

Process Termination (Recovery)

Terminate or restart one controller process (for example, **T4**). This releases **R4**, allowing the other waiting processes to continue and breaking the deadlock cycle.

4. Suggest a Redesign to Prevent Similar Deadlocks

Use a **fixed resource allocation order**.

Example:

$R1 \rightarrow R2 \rightarrow R3 \rightarrow R4$

Every controller must request junctions in this order only.

This removes the circular wait condition and prevents future deadlocks.

Conclusion

Deadlock occurs because all four necessary conditions are present. A fixed resource allocation order and proper recovery techniques can effectively prevent and resolve deadlocks.

Question 10 – University Examination Server Resource Allocation

1. Explain How Banker's Algorithm Can Be Applied

Banker's Algorithm is a **deadlock avoidance algorithm**. Before allocating CPU, memory, or database connections to a student session, the operating system checks whether granting the request will keep the system in a **safe state**.

If the allocation is safe, the request is approved; otherwise, it is postponed until sufficient resources are available.

2. Describe the Information Required Before Granting a Request

The operating system requires the following information:

- **Available Resources:** Number of free CPU units, memory, and database connections.
- **Maximum Resource Requirement:** Maximum resources each student session may require.
- **Current Allocation:** Resources already assigned to each session.
- **Need Matrix:** Remaining resources required by each session.

Need is calculated as:

Need = Maximum – Allocation

3. Explain How the System Determines a Safe State

The system simulates the requested allocation and checks whether all student sessions can complete successfully in some order.

If every session can finish and release its resources, the system is in a **safe state**, and the request is granted.

If no safe sequence exists, the request is denied temporarily to avoid deadlock.

4. Discuss One Limitation of Banker's Algorithm

A major limitation is that it requires the **maximum resource requirement of every process to be known in advance**. In large-scale cloud-based examination systems, resource demands change dynamically as thousands of students log in simultaneously.

Additionally, checking every request increases computation overhead, making the algorithm less practical for highly dynamic environments.

Conclusion

Banker's Algorithm effectively prevents deadlocks by allocating resources only when the system remains in a safe state. Although it improves system reliability, its requirement for prior knowledge of maximum resource needs limits its use in large cloud-based systems.